

Final Review

All About the Final Exam

First and foremost, the final exam will be held on **Thursday 5/23/2024 from 3-5pm** in **Kent 107**. The exam period will be **110 minutes** long. Outside resources will **not be allowed** during the exam. The exam is **cumulative** and may include material from lecture, homework, and discussion worksheets.

Including the **list of topics from the midterm review packet**, the following lists topics that may be on the final exam. The brackets contain relevant chapter numbers in DPV.

1. Graphs [4]

- Shortest Paths [4]
 - Breadth-First Search [4.2]
 - Dijkstra's Algorithm [4.4, 4.5]

2. Greedy algorithms

- Exchange arguments / "Greedy stays ahead"
- Minimum spanning trees [5.1]
 - Kruskal's algorithm [5.1.3]
 - Prim's algorithm [5.1.5]
 - The Cut property [5.1.2]
 - Union-find / disjoint-sets datastructure [5.1.4]
- Optimal prefix free encodings [5.2]
 - Huffman trees

3. Dynamic programming

- Shortest path in graphs with negative edge weights: Bellman-Ford
 - Bellman-Ford [4.6] *see also lecture 15, 16*
 - Shortest / longest path in DAGs [6.1]
 - Knapsack [6.4]
- Edit distance [6.3]

4. Linear programming and network flow [7]

- Linear programming basic definitions [7.1]
- Dual linear program [7.4]
 - Via Von Neumann's minimax theorem *see lecture 19*

- Zero-sum games [7.5]
- Maximum flow [7.2]
 - Basic definitions
 - Ford-Fulkerson
 - Maxflow-mincut

5. NP reductions

- Basic definitions [8.1, 8.2]
 - Search problems, polynomial time reductions, the class P, NP, NP-Complete [8.2]
 - What a polynomial time reduction implies about hardness [8.2]
- Reductions between NP-complete problems. [8.3]
 - $\text{SAT} \rightarrow 3\text{SAT}$
 - $3\text{SAT} \rightarrow \text{INDEPENDENT SET}$
 - $3\text{SAT} \rightarrow \text{C-3SAT}$
 - $\text{C-3SAT} \rightarrow \text{ZOE}$
 - $\text{ZOE} \rightarrow \text{SUBSET SUM}$
 - $\text{SUBSET SUM} \rightarrow \text{KNAPSACK}$

Tips on Studying

Here are some studying tips for the final.

- It's helpful to start in a top-down manner. Review topics in the course at broad and understand how the topics each relate to one another. Prioritize topics by which were most difficult / mysterious. Try not to spend too much time reviewing any one topic. See for example, this [Stack Exchange thread](#).
- Everything covered in this course is fair game for the final; review lecture notes, discussion solutions, past homework questions, and review packets.
- The course textbook *Dasgupta, Papadimitriou & Vazirani* is an excellent resource. The course lecture notes also follow the textbook rather closely.
- Review your previous homeworks. If you would like more feedback on how to answer a question, stop by office hours or post on EdStem. If you did not receive E in a problem, try to understand what would it take to make it an E. If you did receive an E, try to articulate what the main idea of the problem was and how could you make your answer shorter. During the exam you won't have time to write multiple pages for a proof.
- Review previous discussion section worksheets. They are a good source of practice problems.
- Review your notes, and discuss your understanding with friends / course partners. Explaining how you understand an idea to someone else is the best way to test your knowledge.

- Practice with problems in this packet, as well as the exercises in DPV.
- Most importantly, try your best and on the day of the exam, get a good night's rest and remember that all the problems are meant to be have an elegant solution. If you are going over the sixth edge case, there's probably an easier solution.

PROBLEM 1 (Odd Paths). In this question, we help Ruimin find odd paths.

Ruimin is really interested in **odd-length** paths. Formally, he would like to solve the following problem.

- **Input:** An unweighted, directed graph $G = (V, E)$, and two vertices $s, t \in V$.
- **Output:** Either (1) an s - t path p in G that has an **odd** number of edges, or (2) NO if none exist.

Jeremy thinks he can help, and so he's written a program $\mathcal{O}$ which determines whether there is **any** path from s to t in G . Formally, his program satisfies

- **Input:** An unweighted, directed graph $G' = (V', E')$, and two vertices $s', t' \in V'$.
- **Output:** Either (1) an s' - t' path p' in G' , or (2) NO if none exist.

Our goal will be to help Ruimin use Jeremy's program to solve this problem.

- (a) Given a graph $G = (V, E)$ and two vertices $s, t \in V$. Fill in the blanks describing how Ruimin should construct a new graph $G' = (V', E')$ and pick two vertices $s', t' \in V'$ such that the following holds. Running $\mathcal{O}(G', s', t')$ finds an s' - t' path if and only if there is an odd-length path from s to t in G .

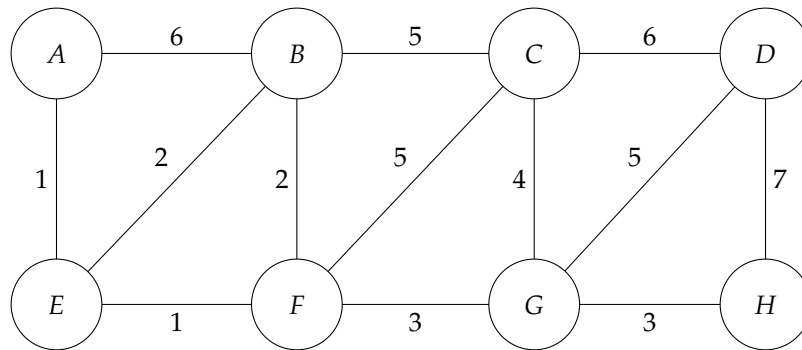
Given $G = (V, E)$ and $s, t \in V$, we construct G' as follows...

We then assign $s' =$ and $t' =$

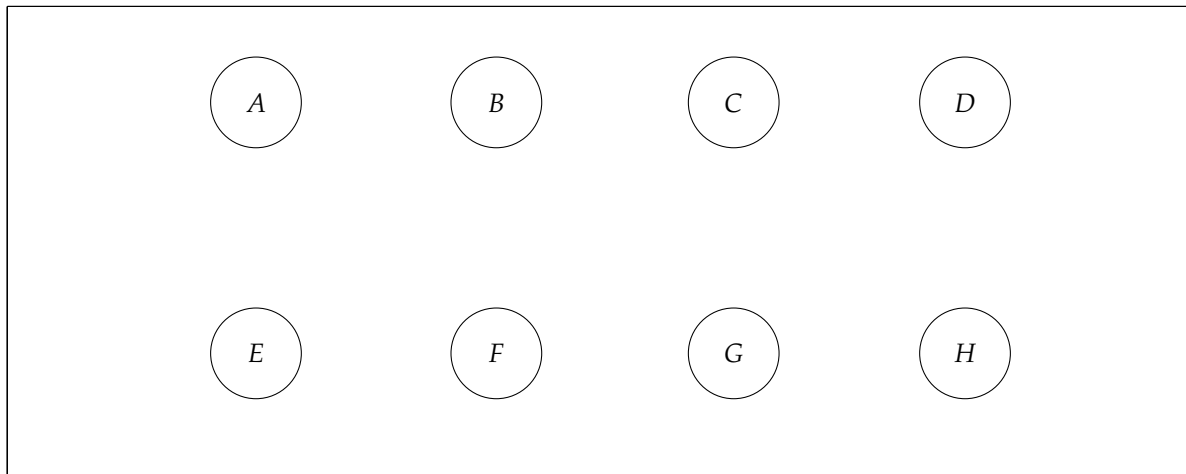
- (b) Using part (a), describe how Ruimin should solve the problem of finding odd paths.

- (c) Finally, let's prove that Ruimin's solution work. Show that $\mathcal{O}(G', s', t')$ finds an s' - t' path if and only if there is an odd-length path from s to t in G

PROBLEM 2 (Running Kruskals). Consider the following weighted graph.



(a) In the diagram below, draw a minimum spanning tree of the given graph.



The total weight of the minimum spanning tree is

- (b) Suppose we run Kruskal's algorithm to compute a minimum spanning tree of the given graph. In what order are edges added? In the diagram below, fill in each box with the edge added, then give a cut which justifies its addition via the cut property.

$e_1 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_2 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_3 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_4 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_5 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_6 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_7 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_8 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)

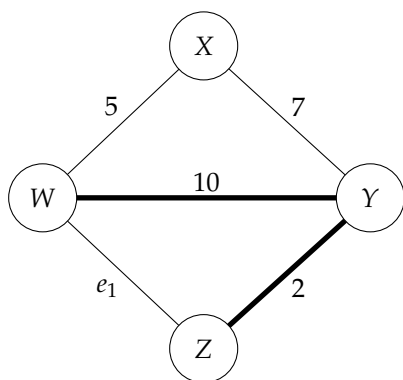
(c) Repeat the same task except now running Prim's algorithm.

$e_1 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_2 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_3 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_4 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_5 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_6 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)
$e_7 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)	$e_8 = \{ \quad , \quad \}$ (A)(B)(C)(D) (E)(F)(G)(H)

PROBLEM 3 (Happy Little Trees). Consider the following graphs. All e_i can have weight varying between 1 and 100 inclusive. For each bolded edge below, mark exactly one of ABCD:

- *A* if for all possible values of e_i 's, **every MST** contains the bolded edge.
- *B* if for all possible values of e_i 's, **only some MSTs** (at least one, but not all) contain the bolded edge.
- *C* if for all possible values of e_i 's, **no MST** contains the bolded edge
- *D* if **none of the above**.

(a) Consider the following graph.



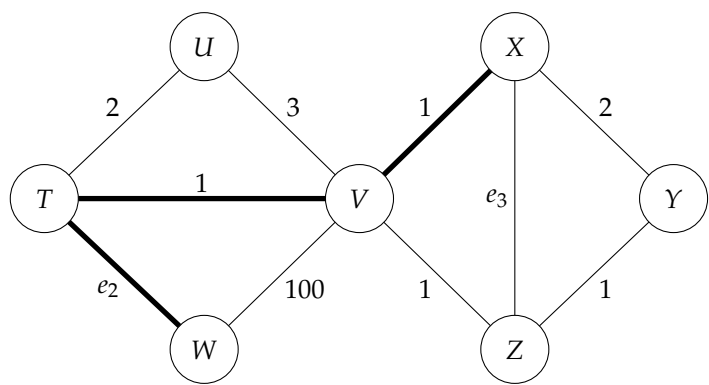
The edge $\{W, Y\}$

☐ A ☐ B ☐ C ☐ D

The edge $\{Z, Y\}$

☐ A ☐ B ☐ C ☐ D

(b) Consider the following graph.



The edge $\{T, V\}$

☐ A	☐ B	☐ C	☐ D
-------------------------	-------------------------	-------------------------	-------------------------

The edge $\{T, W\}$

☐ A	☐ B	☐ C	☐ D
-------------------------	-------------------------	-------------------------	-------------------------

The edge $\{V, X\}$

☐ A	☐ B	☐ C	☐ D
-------------------------	-------------------------	-------------------------	-------------------------

PROBLEM 4 (Huffman Encodings). Let's practice a bit with Huffman encodings.

Consider the Huffman encoding for an alphabet of characters $c_1, \dots, c_n$ such that $n = 2^k$ for some integer $k \geq 1$, with respective frequencies $f_1, \dots, f_n$. For each of the following characters, provide as a function of n the minimum and maximum possible codeword length for all sets f_i such that

$$0 < f_1 \leq \dots \leq f_n < 1.$$

(a) The most frequent character, c_n

Minimum:

Maximum:

(b) The least frequent character, c_1

Minimum:

Maximum:

PROBLEM 5 (Number Cover). In this question, we use greedy to cover some numbers

We are given a set S of n points on the real line. The points are given in **sorted order**. We want to find another set of points C of **minimum size** such that every point in S is **at most distance 1** from at least one point in C . Note that C **need not be a subset of** S . We would like a greedy algorithm that runs in time polynomial with respect to n , and finds an optimal solution.

For example, given the points $S = \{2.7, 3.2, 3.6, 4, 4.9, 5.2\}$, a possible solution is $\{3.5, 5\}$, and there is no smaller solution.

(a) Briefly describe your greedy algorithm (using 3 sentences or less).

(b) What is the running time of your algorithm?

(c) Finally, we prove correctness using an *exchange argument*. Fill in the blanks below to yield a proof.

Consider two solutions: $\mathcal{O}$ that is

and $\mathcal{A}$ given by

Let us suppose (main assumption of an exchange argument)

. Because they

differ, we can pick $x_1 \in \mathcal{A}$ and $x_2 \in \mathcal{O}$ such that

Now, swap $x_1 \in \mathcal{A}$ with $x_2 \in \mathcal{O}$ to create $\mathcal{O}'$. We argue that

and

. This follows as...

Repeating this argument inductively, we can conclude that

hence $\mathcal{A}$ is optimal. $\square$

PROBLEM 6 (Dynamic Programming Orders). The TAs are trying to use dynamic programming and they need your help to implement the algorithm!

Our DP algorithm has two arrays as input: $X[1, \dots, n]$ and $Y[1, \dots, n]$. It defines the subproblems $DP[i, j]$ for each $i, j \in [n]$, and uses the following recurrence relation

$$DP[i, j] = \min \begin{cases} E[i-1, j-1] + 1 & \text{if } X[i] = Y[j-1] \\ E[i-2, \lceil j/2 \rceil] + 3 & \text{if } X[i] = Y[j] \\ E[i-1, j+1] + 1 & \text{if } X[i+1] = Y[j+5] \end{cases}$$

- (a) Fill in the blanks for the following pseudocode for the DP algorithm. It is OK for the code to go out of bounds on DP , that is, we assume that $DP[i, j]$ is computed correctly for $i, j \in \{-2, -1, 0, n+1, n+2\}$.

```

for _____ from 1 to n do {

    for _____ from 1 to n do {

         $DP[i, j] = \min \begin{cases} E[i-1, j-1] + 1 & \text{if } X[i] = Y[j-1] \\ E[i-2, \lceil j/2 \rceil] + 3 & \text{if } X[i] = Y[j] \\ E[i-1, j+1] + 1 & \text{if } X[i+1] = Y[j+5] \end{cases}$ 

    }

}

```

- (b) Suppose our goal is to compute $DP[n, n]$ in polynomial time. What is the smallest memory with which you can implement the above algorithm?

- (c) Describe your implementation in a couple of sentences

PROBLEM 7 (Messaging Friends). Deven's phone is quite rusty.

While Deven is texting his friends, it intermittently inserts spurious characters into his messages. To cope with these characters, Deven repeats his message twice when texting his friends. We will devise an algorithm so that Deven's friends will understand his text messages.

Formally, call a string S a *valid message* if S consists of some string w repeated twice. That is, S is w concatenated with w for some string w .

Deven's friends receive the input string $x[1, \dots, n]$. We will use dynamic programming to design an algorithm that finds the minimum number of character deletions required to make x into a *valid message*. Our algorithm should require at most $O(n^3)$ time.

(a) Briefly describe the main idea behind your algorithm.

(b) Define the subproblem by filling the following blanks.

We will define a recurrence relation $DP(\cdot)$ using the parameters which will

range between . For a fixed choice of parameters, the recurrence relation represents... (briefly describe its meaning)

(c) Now, define the recurrence relation which computes the subproblem.

The base case for the recurrence relation is given by

The recursive case for the recurrence relation is given by

Finally, to compute a solution to this problem, we output

(d) Describe your algorithm using pseudocode.

(e) Finally, prove that your algorithm is correct.

PROBLEM 8 (Zero-sum Games). Konstantinos and Lorenzo are playing a game. A *zero-sum* game.

Konstantinos and Lorenzo are playing a game where the payoff is given by the matrix $\mathbf{M}$. Here, $\mathbf{M}_{ij}$ denotes the payoff that Konstantinos receives if he plays strategy i and Lorenzo plays strategy j .

$$\mathbf{M} = \begin{pmatrix} 4 & 1 \\ 2 & 5 \end{pmatrix}$$

Let's write a linear program which finds a strategy that maximizes Konstantinos's payoff.

(a) Fill in the blanks.

The variables of linear program are x_1, x_2 and p where x_i and p denote...

(b) Write a linear program maximizing Konstantinos's payoff

(c) Find a substitution of variables in your LP for part (b), and eliminate x_2 . Write the resulting LP.

(d) Draw the feasible region for your LP in p and x_1 in part (c).

(e) Find an optimal solution to the your LP in part (b). What is the maximum value that Konstantinos can achieve in this game if both Konstantinos and Lorenzo play optimally?

Optimal Value

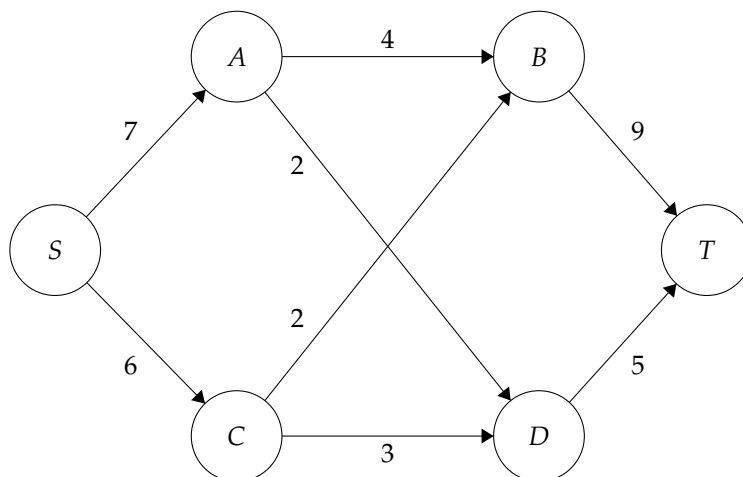
(f) Now, take the dual of your LP in part (b).

What does your dual compute?

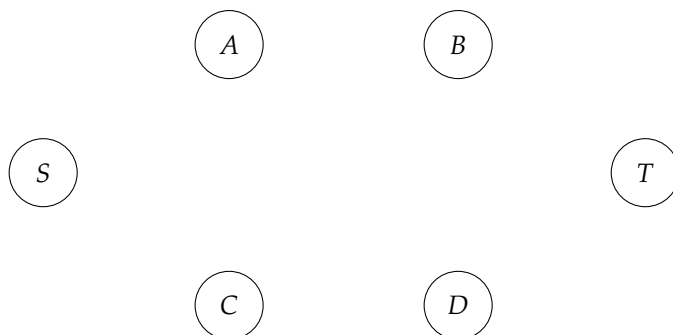
- (g) Finally, provide an assignment to the dual variables showing that the solution you gave in part (e) is indeed optimal. What fact about LPs allows you to conclude that your solution is optimal?

PROBLEM 9 (Practice with Ford-Fulkerson). This is taken from DPV exercise 7.17

Consider the following flow network



(a) Find a maximum flow, and draw the resulting residual graph in the diagram below.



(b) Give a minimum cut of the graph specified as two subsets of vertices.

(c) A *bottleneck edge* is an edge where increasing its capacity results in an increase in the maximum flow. List all bottleneck edges of the graph above.

For the following question, answer (T) rue or (F) alse. If the answer is true, provide a brief explanation why. Otherwise, give a simple counter example.

(d) True or false?

Every flow network has a bottleneck edge.

(e) True or false?

Every flow network has an edge such that decreasing its capacity also decreases the maximum flow size.

(f) True or false?

Maximum flow is solvable in time polynomial with respect to the size of the input.

(g) True or false?

Ford-Fulkerson solves maximum flow in polynomial time.

PROBLEM 10 (WcDonalds). In this question, we will see how LPs are used to approximate discrete problems. You're an executive at the premiere fast food chain WcDonalds, and you've been tasked with expanding the business into Chicago.

We can think of Chicago as a rectangular city divided into $m \times n$ blocks. For each block (i, j) , it costs $r_{ij} > 0$ dollars to rent a location at that block. You would like to setup a restaurant so that for every block in the city, either there is a WcDonalds within the block, or there is a neighboring block. Note that every block has four neighboring blocks.

(a) For every block (i, j) we will create a decision variable x_{ij} where

$$x_{ij} = \begin{cases} 1 & \text{if the optimal solution places a WcDonalds at block } (i, j) \\ 0 & \text{otherwise} \end{cases}$$

What is the objective function?

--

(b) What are the constraints?

Constraint	What does the constraint enforce?

- (c) Suppose we transform the LP provided in parts (a) and (b) by *relaxing* the decision variables to be continuous between the interval $0 \leq \hat{x}_{ij} \leq 1$. We will call this the *fractional LP*.

Provide an interpretation for the fractional LP.

- (d) Argue that the optimal objective value for the *fractional LP* is always at most the optimal objective value for the integral LP provided in parts (a) and (b).

- (e) Given an *optimal solution* to the fractional LP $\hat{x}_{ij}$, suppose we *round* $\hat{x}_{ij}$ to the integral assignment

$$z_{ij} = \begin{cases} 1 & \text{if } \hat{x}_{ij} \geq 1/5 \\ 0 & \text{otherwise} \end{cases}$$

Can you provide a bound on the objective value of the integral assignment z_{ij} , and the objective value of the fractional solution $\hat{x}_{ij}$? What can you conclude about the objective of the rounded solution z_{ij} relative to the optimal integral objective?

PROBLEM 11 (Public Funds). For this question, we consider the following problem PUBLIC FUNDS

You are building a new fence for your mansion.

- You have m bank accounts at your disposal to use to pay for your fence.
- Each account $i \in [m]$ has a balance of b_i .
- There are n different options for your fence.
- Each fence style $j \in [n]$ costs c_j dollars.
- You would like to withdraw from at most k of the bank accounts to build the fence. Due to UC accounting rules, if you use a particular bank account, you must use the whole balance (all b_i dollars).

Determine whether it is possible to exactly pay for some fence j . That is, whether there is a $j \in [n]$ such that you can withdraw exactly c_j dollars given the bank account balances $b_1, \dots, b_m$ and fence costs $c_1, \dots, c_n$ and k . If so, return the corresponding choice of fence and set of bank accounts that you withdraw from.

As it turns out PUBLIC FUNDS is **NP**-Complete. Our goal will be to show this by playing complexity madlibs.

(a) Fill in the blanks to demonstrate that PUBLIC FUNDS is in **NP**.

To show that the problem

is in **NP**, we will exhibit a

for the problem. Given an instance

$\Phi =$

and a candidate solution

$s =$

, our verifier does the following...

- (b) Now, fill in the blanks to define a polynomial time reduction to PUBLIC FUNDS in order to show that it is **NP-Complete**. For this problem, you may assume that the following problems are also **NP-Complete**: 3SAT, INDEPENDENT SET, ZOE, SUBSET SUM, KNAPSACK.

We will exhibit a polynomial time reduction from the problem to the problem

. Given an instance Φ of , we construct an

instance Ψ of as follows...

Next, given a solution of Ψ , we show how to construct a solution of Φ as follows...

We will now show that Ψ has a solution if and only if Φ has a solution. Suppose Ψ has a solution...

- (c) Finally, show that your reduction is indeed a polynomial time reduction.

We first show that our reduction runs in polynomial time. Note that...

We then show that Ψ has a solution if and only if Φ has a solution. Suppose Ψ has a solution, then...

Finally, suppose Φ has a solution. We have...

as required.

□